

Universidade — Disciplina de Engenharia de Software

ConversaJá

Sistema Web de Chat em Tempo Real

Documentação Completa do Projeto

Parte I — Engenharia de Requisitos, Escopo e Modelagem (Trabalho 1)

Parte II — Implementação, Qualidade e Implantação (Trabalho 2)

[Repositório: github.com/Potinho/conversaja](https://github.com/Potinho/conversaja)

Aluno: Vinícios Isael de Oliveira

Professor: Vinícius Nordi Esperança

Entrega Parte I: 24/04/2026 • Parte II: 19/06/2026

Sumário

Sumário	2
1. Documento de Visão e Definição de Escopo	5
1.1. Contexto do problema	5
1.2. Para quem o sistema existe e por que é necessário	5
1.3. Stakeholders	5
1.4. Objetivos do sistema	5
1.5. Definição de escopo	6
1.6. Restrições	6
1.7. Critérios de sucesso	7
2. Estudo de Viabilidade	8
2.1. Viabilidade técnica	8
2.2. Viabilidade econômica	8
2.3. Viabilidade operacional	8
2.4. Riscos iniciais e mitigação	8
3. Especificação de Requisitos de Software (SRS)	10
3.1. Descrição geral do sistema	10
3.2. Glossário	10
3.3. Requisitos funcionais	11
3.4. Requisitos não funcionais	12
3.5. Regras de negócio	13
3.6. Restrições do sistema	14
4. Modelagem do Sistema	15
4.1. Diagrama de casos de uso	15
4.2. Descrição textual dos casos de uso	15
UC01 — Entrar no sistema	16
UC03 — Criar sala pública	16
UC04 — Enviar mensagem em tempo real	16
UC08 — Moderar sala (remover mensagem)	17
4.3. Diagrama de classes conceitual	18
4.4. Diagrama de sequência (funcionalidade crítica)	18
4.5. Diagrama de estados (complementar)	19
5. Validação e Rastreabilidade	21
5.1. Sessão de revisão interna	21
5.2. Matriz de rastreabilidade	21
6. Implementação do Sistema	23
6.1. Do Trabalho 1 ao Trabalho 2	23
6.2. Tecnologias e organização do código	23
6.3. Repositório e versionamento	23
7. Projeto de Interfaces (Protótipos)	25

7.1. Tela de entrada	25
7.2. Lobby de salas	25
7.3. Sala de conversa	26
7.4. Painel administrativo	26
8. Arquitetura do Sistema	28
8.1. Estilo arquitetural e justificativa	28
8.2. Visão geral de componentes	28
8.3. Componentes principais	28
Frontend (Angular)	28
Backend (NestJS)	29
Pacote compartilhado (@conversaja/shared)	29
8.4. Formas de comunicação	29
8.5. Fluxo crítico — envio de mensagem	30
8.6. Persistência — durável vs. efêmero	31
8.7. Decisões arquiteturais relevantes	32
9. Qualidade e Testes	33
9.1. Níveis de teste	33
9.2. Atributos de qualidade priorizados	33
9.3. Casos de teste prioritários	33
9.4. Teste de fluxo completo (end-to-end)	34
9.5. Execução dos testes	34
10. Implantação e CI/CD	35
10.1. Implantação por terceiros (Docker Compose)	35
10.2. Integração contínua (CI)	35
11. Rastreabilidade da Implementação	36
12. Considerações Finais	37

PARTE I

Engenharia de Requisitos, Escopo e Modelagem

Trabalho 1

1. Documento de Visão e Definição de Escopo

1.1. Contexto do problema

Em diversos contextos — eventos, salas de aula, grupos de estudo, comunidades online e pequenas equipes — surge a necessidade de comunicação textual instantânea entre várias pessoas reunidas em torno de um mesmo assunto. As ferramentas mais populares para esse fim (mensageiros e plataformas corporativas) frequentemente exigem cadastro completo, instalação de aplicativos, criação de grupos ou vínculo a redes sociais, o que introduz fricção e exclui participação rápida e pontual.

O problema central é a ausência de um espaço web leve, de acesso imediato, em que uma pessoa possa entrar pelo navegador, identificar-se com um apelido e começar a conversar em tempo real em salas temáticas, sem barreiras de instalação ou cadastro complexo. O ConversaJá nasce para atender exatamente essa lacuna: comunicação em tempo real, simples e de baixa fricção.

1.2. Para quem o sistema existe e por que é necessário

O sistema existe para pessoas que precisam conversar coletivamente e de forma instantânea por períodos curtos ou recorrentes, sem o peso de uma plataforma robusta. É necessário porque reduz a barreira de entrada (acesso por navegador, sem instalação), organiza a conversa por salas temáticas e oferece a sensação de presença por meio de lista de participantes online e atualização em tempo real — algo que e-mail, fóruns ou ferramentas pesadas não entregam com a mesma agilidade.

1.3. Stakeholders

Os principais interessados no sistema são:

Stakeholder	Interesse / Papel
Usuário final (Participante)	Entra no sistema, escolhe um apelido, participa de salas e troca mensagens em tempo real.
Moderador de sala	Usuário com permissões adicionais em uma sala: pode remover mensagens e expulsar participantes para manter a ordem.
Administrador do sistema	Gerencia salas oficiais/globais, acompanha o uso e mantém a integridade do serviço.
Mantenedor / Equipe de desenvolvimento	Constrói, hospeda e evolui o sistema ao longo da disciplina (Trabalho 2).
Cliente / Avaliador (docente)	Avalia o alinhamento entre problema, requisitos e solução proposta.

1.4. Objetivos do sistema

- Permitir comunicação textual em tempo real entre múltiplos usuários reunidos em salas.

- Oferecer acesso imediato pelo navegador, identificando o usuário por um apelido, sem cadastro complexo.
- Organizar as conversas em salas públicas temáticas, criáveis pelos próprios usuários.
- Transmitir a sensação de presença por meio de lista de participantes online e indicadores de atividade.
- Disponibilizar mecanismos básicos de moderação para preservar um ambiente saudável.

1.5. Definição de escopo

A delimitação explícita do escopo é um dos pilares deste trabalho. A tabela abaixo separa, de forma objetiva, o que está dentro e o que está fora do projeto.

Dentro do escopo (será feito)	Fora do escopo (não será feito)
• Entrada no sistema com apelido (identificação leve). • Criação e listagem de salas públicas temáticas. • Envio e recebimento de mensagens de texto em tempo real. • Lista de participantes online por sala. • Histórico recente de mensagens por sala. • Indicador de “digitando...” e avisos de entrada/saída. • Moderação básica (remover mensagem, expulsar usuário). • Painel administrativo simples para salas oficiais.	• Chamadas de voz e vídeo. • Compartilhamento de arquivos, imagens ou streaming. • Mensagens privadas individuais (DM) entre dois usuários. • Criptografia ponta-a-ponta. • Aplicativos nativos para iOS/Android. • Integração com redes sociais e login social. • Bots, assistentes de IA e tradução automática. • Monetização, pagamentos ou planos pagos.

Justificativa: o foco do projeto está na qualidade da engenharia de uma funcionalidade central (conversa em tempo real), e não na quantidade de recursos. Itens fora do escopo ou aumentam muito a complexidade técnica (voz/vídeo, criptografia ponta-a-ponta) ou desviam do problema essencial (redes sociais, pagamentos), sendo incompatíveis com o prazo e a equipe da disciplina.

1.6. Restrições

Tipo	Restrição
Tecnológicas	Aplicação exclusivamente Web, executada em navegadores modernos. Back-end em Node.js com o framework NestJS, expondo a comunicação em tempo real via WebSocket Gateways; front-end em Angular, conectando-se por meio da biblioteca Socket.IO. Todas as tecnologias são gratuitas/de código aberto e o sistema deve executar em servidor de capacidade modesta.

Tipo	Restrição
Legais	Conformidade com a LGPD: coleta mínima de dados, ausência de dados sensíveis, transparência sobre o uso das informações e tráfego sobre conexão segura (HTTPS/WSS).
Organizacionais	Prazo limitado ao cronograma da disciplina; equipe pequena com conhecimento em formação; o sistema deve ser efetivamente implementável no Trabalho 2.

1.7. Critérios de sucesso

- Um usuário sem treinamento consegue entrar e enviar a primeira mensagem em menos de 60 segundos.
- Mensagens são entregues aos demais participantes da sala em menos de 1 segundo, em condições normais de uso.
- O sistema suporta pelo menos 50 usuários simultâneos em uma sala sem perda perceptível de desempenho.
- O sistema permanece disponível durante toda a sessão de demonstração da disciplina.
- As funcionalidades dentro do escopo são demonstráveis de ponta a ponta no Trabalho 2.

2. Estudo de Viabilidade

Este estudo avalia, antes da construção, se o sistema proposto é razoável sob os aspectos técnico, econômico e operacional, identificando ainda os riscos iniciais e suas estratégias de mitigação.

2.1. Viabilidade técnica

As tecnologias adotadas são maduras, amplamente documentadas e gratuitas. O back-end será desenvolvido em Node.js com o framework NestJS, que expõe a comunicação em tempo real por meio de WebSocket Gateways. O front-end será uma aplicação Angular que se conecta ao servidor utilizando a biblioteca Socket.IO, responsável pela troca de eventos em tempo real (envio e broadcast de mensagens) sobre WebSocket. Tanto NestJS quanto Angular utilizam a linguagem TypeScript, o que favorece a consistência e a manutenibilidade do código entre back-end e front-end. A persistência das mensagens pode usar um banco de dados leve (por exemplo, SQLite ou PostgreSQL). Os conhecimentos exigidos estão ao alcance da equipe ou são aprendíveis dentro do prazo, o que torna o projeto tecnicamente viável.

2.2. Viabilidade econômica

Não há custos de licença, pois todas as ferramentas previstas são de código aberto. A hospedagem pode ser feita em planos gratuitos ou de baixíssimo custo adequados ao porte do projeto. O principal recurso investido é o tempo da equipe, compatível com a carga horária da disciplina, desde que o escopo seja respeitado. Portanto, o projeto é economicamente viável no contexto acadêmico.

2.3. Viabilidade operacional

Por ser acessível diretamente pelo navegador, sem instalação, o sistema pode ser efetivamente utilizado no contexto descrito (eventos, turmas, grupos). A curva de aprendizado é baixa, e o fluxo “entrar, escolher sala e conversar” é familiar à maioria dos usuários. A operação durante a demonstração depende apenas de um servidor ativo e de conexão à internet, o que é plenamente factível.

2.4. Riscos iniciais e mitigação

Risco	Descrição	Estratégia de mitigação
R1	Pouca familiaridade da equipe com comunicação em tempo real (WebSocket/Socket.IO) e com o framework NestJS.	Realizar uma prova de conceito simples logo no início, antes do desenvolvimento principal, e usar bibliotecas consolidadas com boa documentação.
R2	Crescimento descontrolado do escopo (adição de voz, DM, etc.).	Manter a definição de escopo deste documento como referência e tratar novos recursos como melhorias futuras.

Risco	Descrição	Estratégia de mitigação
R3	Problemas de desempenho com muitos usuários simultâneos.	Limitar a capacidade por sala (regra de negócio), testar com carga simulada e otimizar o broadcast de mensagens.
R4	Conteúdo abusivo ou spam nas salas públicas.	Disponibilizar moderação (remoção de mensagens e expulsão) e validar/sanitizar o conteúdo enviado.
R5	Indisponibilidade da hospedagem na demonstração.	Preparar ambiente de contingência local e testar a implantação com antecedência.

3. Especificação de Requisitos de Software (SRS)

3.1. Descrição geral do sistema

O ConversaJá é uma aplicação Web de chat em tempo real. Após acessar o sistema e informar um apelido, o usuário é levado a um lobby onde visualiza as salas públicas disponíveis, podendo entrar em uma sala existente ou criar uma nova. Dentro de uma sala, o usuário troca mensagens de texto que são entregues instantaneamente a todos os participantes, vê quem está online e recebe avisos de entrada e saída. Salas possuem moderadores, que podem remover mensagens e expulsar participantes; o administrador do sistema gerencia salas oficiais e acompanha o uso geral.

3.2. Glossário

Termo	Definição
Apelido	Nome de exibição escolhido pelo usuário para se identificar no sistema durante a sessão.
Sala	Espaço temático que agrupa participantes e mensagens; pode ser pública ou oficial.
Mensagem	Conteúdo de texto enviado por um participante a uma sala.
Participante	Usuário que entrou em uma sala e pode enviar e receber mensagens.
Moderador	Participante com permissões adicionais em uma sala (remover mensagens, expulsar usuários).
Administrador	Usuário responsável por gerenciar salas oficiais e monitorar o sistema.
Tempo real	Entrega de mensagens aos destinatários quase imediatamente após o envio, sem recarregar a página.
WebSocket	Protocolo de comunicação bidirecional persistente entre cliente e servidor, usado para a troca em tempo real.
NestJS	Framework Node.js (TypeScript) usado no back-end; expõe a comunicação em tempo real por meio de WebSocket Gateways.
Angular	Framework front-end (TypeScript) usado para construir a interface Web do sistema.
Socket.IO	Biblioteca que abstrai a comunicação em tempo real sobre WebSocket, baseada em eventos (emit/broadcast), utilizada por servidor e cliente.
Broadcast	Transmissão de uma mensagem do servidor para todos os participantes de uma sala.
Sessão	Período em que o usuário permanece conectado ao sistema sob um mesmo apelido.

3.3. Requisitos funcionais

Cada requisito é identificado, descrito de forma testável e acompanhado de um critério de aceitação que define quando pode ser considerado atendido.

ID	Requisito	Descrição e critério de aceitação
RF01	Entrar no sistema com apelido	O sistema deve permitir que o usuário acesse informando um apelido válido e único entre os usuários conectados. Critério de aceitação: Dado um apelido válido e disponível, o usuário obtém acesso ao lobby; dado apelido inválido ou em uso, o acesso é negado com mensagem explicativa.
RF02	Listar salas públicas	O sistema deve exibir a lista de salas públicas com nome, tema e quantidade de participantes online. Critério de aceitação: A lista reflete as salas existentes e é atualizada quando salas são criadas ou removidas.
RF03	Entrar em uma sala	O sistema deve permitir que o usuário entre em uma sala selecionada. Critério de aceitação: Após entrar, o usuário vê o histórico recente e a lista de participantes; os demais recebem aviso de entrada.
RF04	Criar sala pública	O sistema deve permitir que o usuário crie uma sala pública informando nome e tema. Critério de aceitação: A sala criada passa a aparecer na lista para todos, e o criador entra nela como moderador.
RF05	Enviar mensagem em tempo real	O sistema deve permitir o envio de mensagens de texto à sala atual, entregues a todos os participantes em tempo real. Critério de aceitação: A mensagem enviada aparece para todos os participantes da sala dentro do limite de desempenho e é persistida.
RF06	Receber mensagens em tempo real	O sistema deve exibir as mensagens recebidas dos demais participantes sem recarregar a página. Critério de aceitação: Mensagens de outros participantes aparecem automaticamente na conversa.
RF07	Visualizar usuários online	O sistema deve exibir a lista de participantes atualmente online na sala. Critério de aceitação: A lista é atualizada a cada entrada e saída de participantes.
RF08	Visualizar histórico recente	O sistema deve exibir as últimas N mensagens da sala ao entrar nela. Critério de aceitação: Ao entrar, as últimas N mensagens são exibidas em ordem cronológica.

ID	Requisito	Descrição e critério de aceitação
RF09	Sinalizar “digitando...”	O sistema deve indicar aos demais quando um participante está digitando. Critério de aceitação: O indicador aparece enquanto o participante digita e desaparece quando ele para.
RF10	Notificar entrada e saída	O sistema deve registrar e exibir avisos de entrada e saída de participantes na sala. Critério de aceitação: Entradas e saídas geram uma notificação visível aos participantes.
RF11	Sair da sala / do sistema	O sistema deve permitir que o usuário saia da sala ou encerre a sessão. Critério de aceitação: Após a saída, o usuário é removido da lista de participantes e os demais são notificados.
RF12	Remover mensagem (moderação)	O sistema deve permitir que um moderador remova uma mensagem da sala. Critério de aceitação: Apenas o moderador da sala executa a ação; a mensagem deixa de ser exibida para todos.
RF13	Expulsar usuário (moderação)	O sistema deve permitir que um moderador expulse um participante da sala. Critério de aceitação: Apenas o moderador executa a ação; o participante expulso é removido e impedido de enviar mensagens na sala.
RF14	Gerenciar salas oficiais (admin)	O sistema deve permitir que o administrador crie, edite e remova salas oficiais. Critério de aceitação: As alterações feitas pelo administrador refletem-se na lista de salas para todos.
RF15	Monitorar o sistema (admin)	O sistema deve apresentar ao administrador indicadores de uso (salas ativas e usuários online). Critério de aceitação: O painel exibe a quantidade de salas ativas e de usuários conectados.

3.4. Requisitos não funcionais

Os requisitos não funcionais trazem, sempre que possível, critérios mensuráveis ou observáveis.

ID	Categoria	Critério mensurável / observável
RNF01	Desempenho	Em condições normais (até 50 usuários por sala), pelo menos 95% das mensagens devem ser entregues aos demais participantes em menos de 1 segundo.
RNF02	Capacidade	O sistema deve suportar no mínimo 50 usuários simultâneos por sala e 200 usuários no total sem degradação perceptível.

ID	Categoria	Critério mensurável / observável
RNF03	Disponibilidade	Durante as janelas de operação previstas (incluindo a demonstração), a disponibilidade deve ser de pelo menos 99%.
RNF04	Usabilidade	Um usuário sem treinamento deve conseguir entrar e enviar a primeira mensagem em até 60 segundos; a interface deve estar em português (pt-BR).
RNF05	Compatibilidade	O sistema deve funcionar nas duas versões mais recentes de Chrome, Firefox e Edge, e ser responsivo para telas a partir de 360px de largura.
RNF06	Segurança	Todo o tráfego deve ocorrer sobre HTTPS/WSS, e o conteúdo das mensagens deve ser sanitizado para evitar injeção de scripts (XSS).
RNF07	Privacidade (LGPD)	O sistema deve coletar o mínimo de dados (apelido), não armazenar dados pessoais sensíveis e informar ao usuário como os dados são utilizados.
RNF08	Confiabilidade	Em caso de queda de conexão, o cliente deve tentar reconectar automaticamente em até 5 segundos e restaurar a sessão na mesma sala.
RNF09	Manutenibilidade	O código deve ser modular e documentado, seguindo um padrão de estilo, de forma que uma nova funcionalidade simples possa ser adicionada sem alterar módulos não relacionados.
RNF10	Portabilidade	A aplicação deve ser acessível sem instalação, executando-se inteiramente no navegador; o servidor deve rodar em ambiente Linux padrão com o runtime Node.js.

3.5. Regras de negócio

ID	Regra de negócio
RN01	O apelido deve ser único entre os usuários conectados e ter de 3 a 20 caracteres alfanuméricos.
RN02	Somente o moderador de uma sala pode remover mensagens e expulsar participantes dessa sala.
RN03	O usuário que cria uma sala torna-se automaticamente seu primeiro moderador.
RN04	Cada mensagem é limitada a 500 caracteres.
RN05	Mensagens vazias (apenas espaços) não são enviadas.
RN06	Um usuário expulso não pode reingressar na mesma sala, com o mesmo apelido, por 10 minutos.

ID	Regra de negócio
RN07	Salas públicas (não oficiais) sem participantes por mais de 30 minutos podem ser removidas automaticamente.
RN08	Cada sala possui capacidade máxima (padrão de 50 participantes); quando cheia, novos usuários não conseguem entrar.

3.6. Restrições do sistema

- Aplicação exclusivamente Web; não haverá versão nativa para dispositivos móveis.
- Comunicação em tempo real obrigatoriamente via WebSocket, usando Socket.IO (HTTPS/WSS).
- Uso restrito a tecnologias gratuitas e de código aberto.
- Conformidade obrigatória com a LGPD quanto à coleta e ao uso de dados.

4. Modelagem do Sistema

Os modelos UML a seguir complementam a especificação textual, reduzindo ambiguidades. Eles foram construídos em coerência com os requisitos da Seção 3.

4.1. Diagrama de casos de uso

O diagrama apresenta os atores (Usuário, Moderador e Administrador) e as principais funcionalidades do sistema. O Moderador e o Administrador possuem as capacidades do Usuário, acrescidas de suas funções específicas. As relações «include» indicam funcionalidades obrigatoriamente reutilizadas (por exemplo, enviar mensagem inclui estar autenticado e em uma sala).

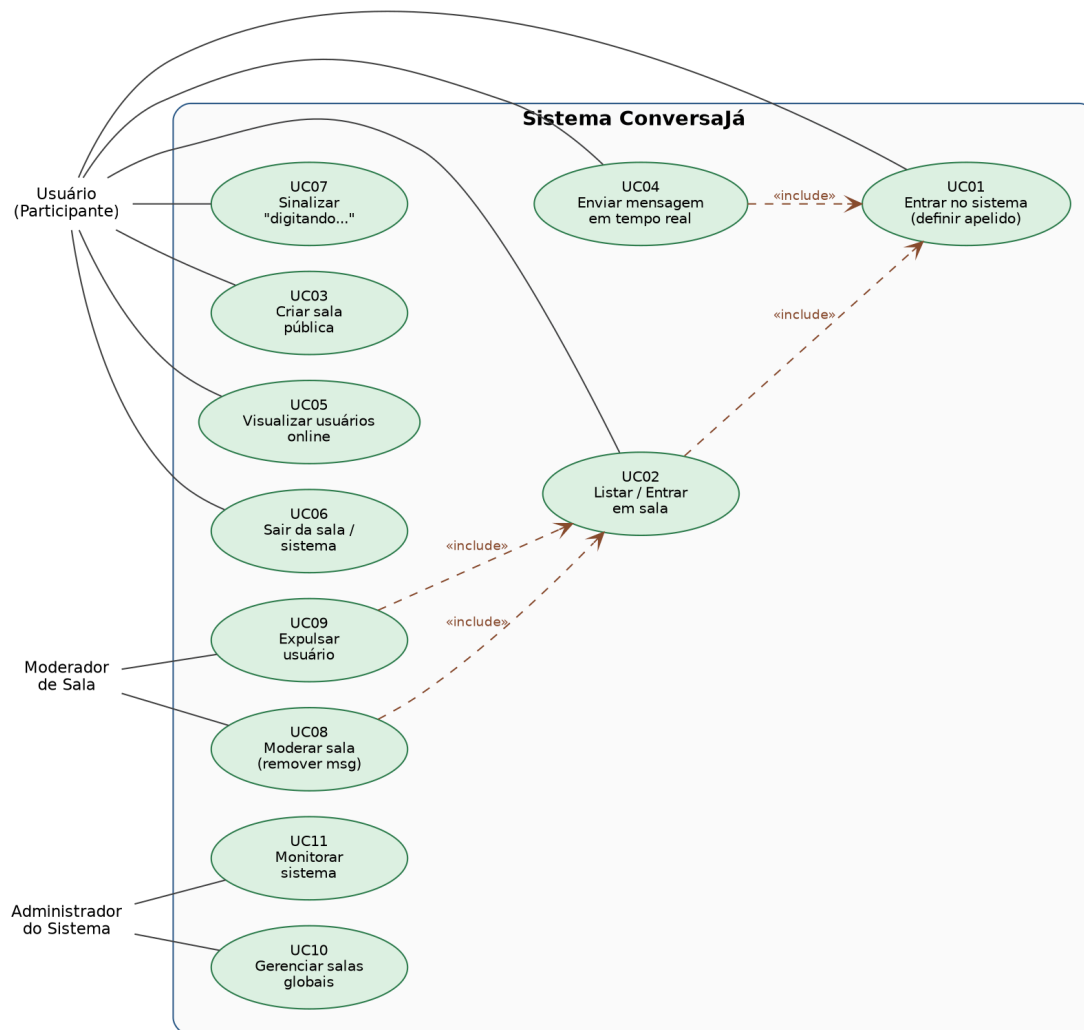


Figura 1 — Diagrama de casos de uso do ConversaJá.

4.2. Descrição textual dos casos de uso

São detalhados a seguir quatro casos de uso representativos, com fluxo principal e fluxos alternativos.

UC01 — Entrar no sistema

Ator principal	Usuário
Pré-condições	O usuário acessou a URL do sistema e está desconectado.
Pós-condições	O usuário está autenticado por apelido e em estado “Online”, no lobby.
Fluxo principal	1. O usuário acessa a página inicial do sistema. 2. O sistema exibe o campo para informar um apelido. 3. O usuário informa o apelido e confirma. 4. O sistema valida o apelido quanto ao formato e à unicidade (RN01). 5. O sistema registra a sessão e direciona o usuário ao lobby (lista de salas).
Fluxos alternativos	FA1 (apelido inválido): no passo 4, se o apelido não respeitar o formato, o sistema exibe mensagem e retorna ao passo 2. FA2 (apelido em uso): no passo 4, se já houver usuário conectado com o mesmo apelido, o sistema solicita outro apelido (retorna ao passo 2).

UC03 — Criar sala pública

Ator principal	Usuário
Pré-condições	O usuário está autenticado (UC01) e no lobby.
Pós-condições	Uma nova sala é criada e listada; o usuário entra nela como moderador.
Fluxo principal	1. O usuário seleciona a opção “Criar sala”. 2. O sistema exibe o formulário com os campos nome e tema. 3. O usuário preenche os campos e confirma. 4. O sistema valida os dados (nome não vazio e dentro do limite). 5. O sistema cria a sala, define o criador como moderador (RN03) e a adiciona à lista pública. 6. O sistema insere o usuário na nova sala (UC02).
Fluxos alternativos	FA1 (dados inválidos): no passo 4, se a validação falhar, o sistema exibe erro e retorna ao passo 2. FA2 (nome duplicado): se já existir sala com o mesmo nome, o sistema solicita outro nome (retorna ao passo 2).

UC04 — Enviar mensagem em tempo real

Ator principal	Usuário (remetente); demais participantes (receptores)
Pré-condições	O usuário está autenticado e dentro de uma sala (UC02/UC03).

Pós-condições	A mensagem é persistida e exibida a todos os participantes da sala.
Fluxo principal	1. O usuário digita o texto da mensagem. 2. O usuário envia a mensagem. 3. O sistema valida a sessão e o conteúdo (RN04, RN05). 4. O sistema persiste a mensagem. 5. O sistema transmite (broadcast) a mensagem a todos os participantes da sala. 6. Os clientes exibem a mensagem em tempo real.
Fluxos alternativos	FA1 (conteúdo inválido): no passo 3, se a mensagem for vazia ou exceder o limite, o sistema a rejeita e avisa o remetente, sem broadcast. FA2 (falha de persistência): no passo 4, se a gravação falhar, o sistema tenta novamente; persistindo a falha, avisa o remetente. FA3 (conexão perdida): se a conexão cair, o cliente tenta reconectar (RNF08) e reenvia mensagens pendentes ao restabelecer a conexão.

UC08 — Moderar sala (remover mensagem)

Ator principal	Moderador
Pré-condições	O moderador está autenticado e em uma sala na qual possui papel de moderador (RN02).
Pós-condições	A mensagem selecionada é removida para todos os participantes.
Fluxo principal	1. O moderador seleciona uma mensagem na sala. 2. O moderador escolhe a ação “Remover”. 3. O sistema verifica se o ator é moderador da sala (RN02). 4. O sistema remove a mensagem e transmite a atualização a todos. 5. Os clientes deixam de exibir a mensagem.
Fluxos alternativos	FA1 (sem permissão): no passo 3, se o ator não for moderador, o sistema nega a ação. FA2 (mensagem inexistente): se a mensagem já tiver sido removida, o sistema informa e atualiza a visão da sala.

4.3. Diagrama de classes conceitual

O diagrama conceitual representa as principais entidades do domínio e seus relacionamentos. Um Usuário pode enviar diversas Mensagens, e uma Sala agrupa diversas Mensagens. A relação entre Usuário e Sala é de muitos-para-muitos e é qualificada pela classe associativa Participação, que registra o papel do usuário na sala (participante ou moderador) e os instantes de entrada e saída. O Administrador é uma especialização de Usuário, com atributos próprios de gestão.

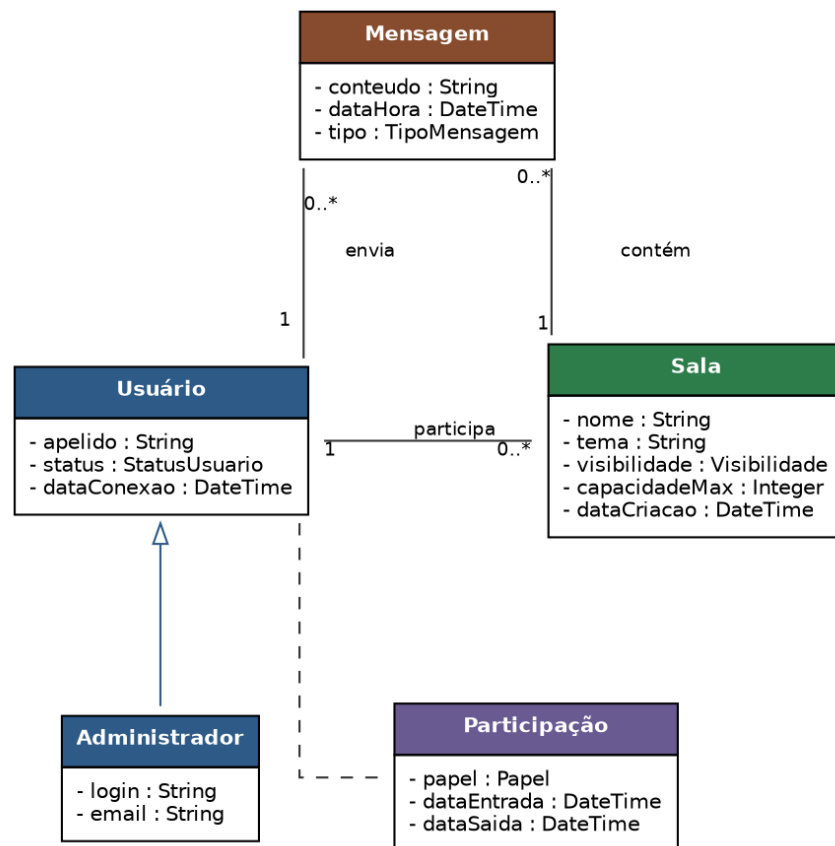


Figura 2 — Diagrama de classes conceitual do domínio.

4.4. Diagrama de sequência (funcionalidade crítica)

A funcionalidade crítica do sistema é o envio de mensagem em tempo real (UC04/RF05). O diagrama de sequência ilustra a interação entre o remetente, os clientes Angular, o servidor NestJS (com Socket.IO) e o banco de dados: a mensagem é validada e persistida no servidor e, em seguida, transmitida por broadcast a todos os participantes, que a exibem imediatamente. O fluxo alternativo de falha de validação também está indicado.

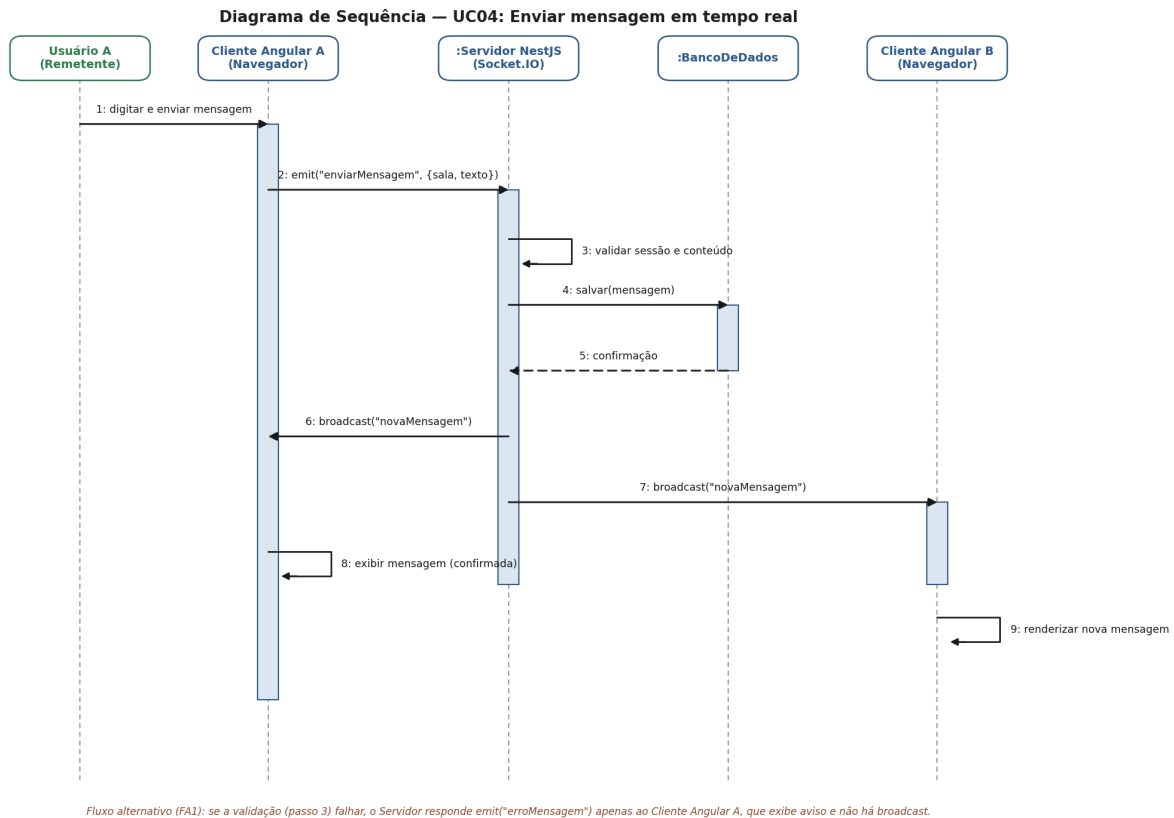


Figura 3 — Diagrama de sequência: envio de mensagem em tempo real (UC04).

4.5. Diagrama de estados (complementar)

Como modelo comportamental complementar, o diagrama de estados descreve o ciclo de vida da sessão de um usuário, desde a conexão e autenticação por apelido até a participação em salas, o estado de inatividade e a desconexão. Ele evidencia as transições relevantes para o comportamento em tempo real do sistema.

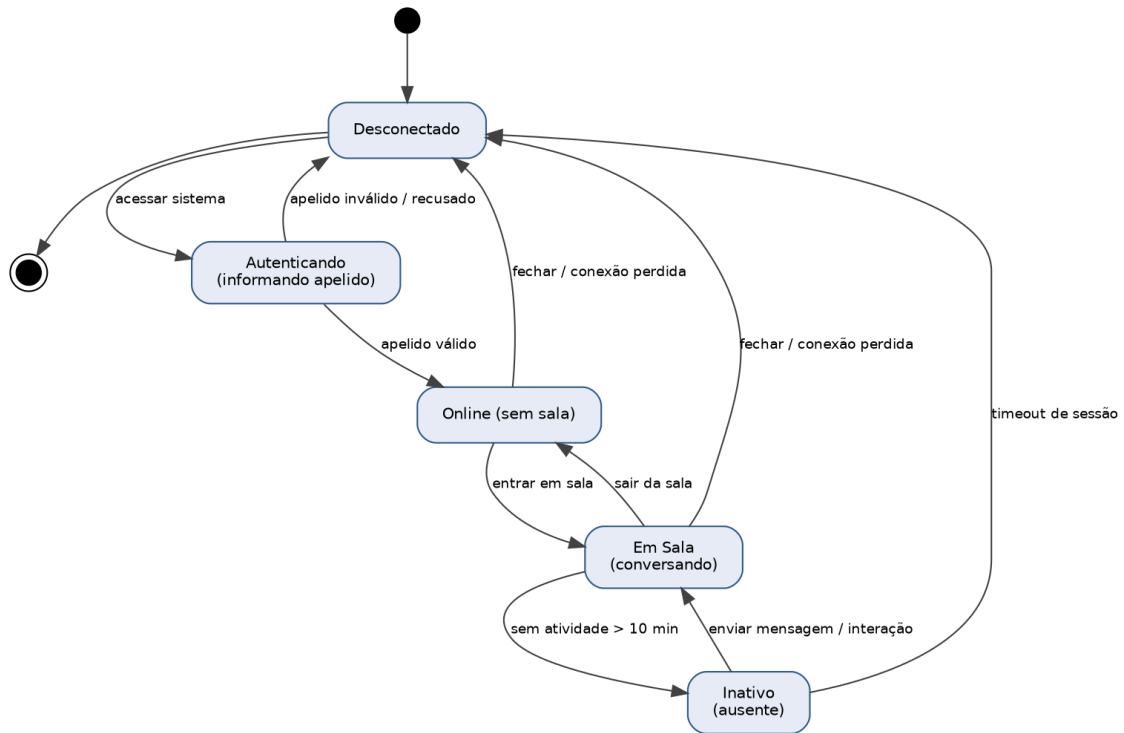


Figura 4 — Diagrama de estados da sessão do usuário.

5. Validação e Rastreabilidade

5.1. Sessão de revisão interna

A equipe realizou uma sessão de revisão interna dos requisitos no formato de leitura estruturada (walkthrough). Os integrantes percorreram, em conjunto, cada requisito funcional e não funcional, confrontando-o com o documento de visão e com os modelos UML. O objetivo foi verificar clareza, ausência de ambiguidade, completude, consistência e testabilidade. Como resultado, foram identificados e corrigidos pontos como: a inclusão de critérios mensuráveis em requisitos não funcionais que estavam genéricos (por exemplo, definição de limites numéricos em RNF01 e RNF02) e o reforço dos critérios de aceitação dos requisitos funcionais para torná-los verificáveis. A tabela a seguir resume o checklist de qualidade aplicado.

Critério	Pergunta de verificação	Resultado
Clareza	Cada requisito é compreensível sem conhecimento adicional?	Atendido
Ausência de ambiguidade	Há uma única interpretação possível para cada requisito?	Atendido
Completude	As funcionalidades do escopo estão todas cobertas por requisitos?	Atendido
Consistência	Os requisitos são compatíveis entre si e com os modelos UML?	Atendido
Testabilidade	Cada requisito possui critério objetivo de aceitação/verificação?	Atendido
Rastreabilidade	É possível ligar cada requisito a casos de uso e/ou diagramas?	Atendido

5.2. Matriz de rastreabilidade

A matriz a seguir liga cada requisito funcional ao(s) caso(s) de uso correspondente(s) e aos modelos (diagramas) que o representam, evidenciando o controle e a evolução dos requisitos.

Requisito (RF)	Caso(s) de uso	Modelo(s) / Diagrama(s)
RF01	UC01	Casos de uso; Estados
RF02	UC02	Casos de uso
RF03	UC02	Casos de uso; Estados
RF04	UC03	Casos de uso; Classes
RF05	UC04	Casos de uso; Sequência; Classes
RF06	UC04	Sequência
RF07	UC05	Casos de uso; Classes

Requisito (RF)	Caso(s) de uso	Modelo(s) / Diagrama(s)
RF08	UC02 / UC04	Classes (Mensagem)
RF09	UC07	Casos de uso
RF10	UC02 / UC06	Casos de uso; Estados
RF11	UC06	Casos de uso; Estados
RF12	UC08	Casos de uso
RF13	UC09	Casos de uso
RF14	UC10	Casos de uso
RF15	UC11	Casos de uso

Esta rastreabilidade garante que toda funcionalidade prevista possui origem em um requisito e representação em pelo menos um modelo, preparando o terreno para a implementação e os testes do Trabalho 2.

PARTE II

Implementação, Qualidade e Implantação

Trabalho 2

6. Implementação do Sistema

6.1. Do Trabalho 1 ao Trabalho 2

Este documento dá continuidade ao Trabalho 1, no qual foram definidos o escopo, os requisitos e os modelos do ConversaJá — um sistema Web de chat em tempo real, com salas temáticas e identificação por apelido, sem cadastro complexo. Aqui o projeto conceitual foi transformado em uma aplicação Web funcional, implantável e testável, mantendo a stack então escolhida: back-end em Node.js com NestJS, front-end em Angular e comunicação em tempo real via Socket.IO. Todas as 15 funcionalidades (RF01–RF15) e as 8 regras de negócio (RN01–RN08) do escopo foram implementadas.

6.2. Tecnologias e organização do código

O sistema é uma aplicação Web completa — interface de usuário, back-end e persistência — escrita inteiramente em TypeScript e organizada em um monorepo com npm workspaces, o que permite compartilhar os contratos (eventos, DTOs, enums e limites de negócio) entre cliente e servidor a partir de uma única fonte de verdade.

Camada	Tecnologia	Responsabilidade
Frontend	Angular (zoneless + signals)	Interface em pt-BR, cliente WebSocket/REST e estado de tela.
Backend	NestJS + Socket.IO	Gateway WebSocket, API REST e regras de negócio.
Persistência	PostgreSQL via TypeORM	Salas e mensagens (a presença online é efêmera).
Compartilhado	@conversaja/shared	Contratos: eventos WebSocket, DTOs, enums e limites.

A estrutura de pastas do monorepo reflete essa separação:

```

conversaja/
├── apps/
│   ├── frontend/  # Angular — UI e cliente WebSocket/REST
│   └── backend/    # NestJS — gateway WebSocket + REST + persistência
├── packages/
│   └── shared/     # contratos compartilhados (fonte única de verdade)
├── docs/           # requisitos, arquitetura, implementação, protótipos
└── .github/        # pipeline de integração contínua (CI)

```

6.3. Repositório e versionamento

O código está versionado em Git desde o início do projeto, hospedado no GitHub:

<https://github.com/Potinho/conversaja>

Foram adotadas boas práticas básicas de versionamento: desenvolvimento de cada funcionalidade em branches próprias, integração via Pull Requests com a verificação automática do CI (lint, testes e build) exigida antes do merge na branch principal, e mensagens

de commit claras. As convenções de commit e de código do projeto estão documentadas no próprio repositório.

7. Projeto de Interfaces (Protótipos)

Antes e em paralelo à implementação foram produzidos protótipos de média fidelidade das telas principais, representando os fluxos de interação do usuário. A implementação final mantém coerência com esses protótipos. As quatro telas a seguir cobrem o fluxo completo do sistema.

7.1. Tela de entrada

Tela inicial de baixa fricção: um único campo de apelido, com validação de formato e unicidade (3 a 20 caracteres alfanuméricos — RN01) e mensagem clara de erro. O rodapé informa, de forma transparente, que apenas o apelido é coletado e que a conexão é segura (RNF06/RNF07). Corresponde ao RF01 / UC01.

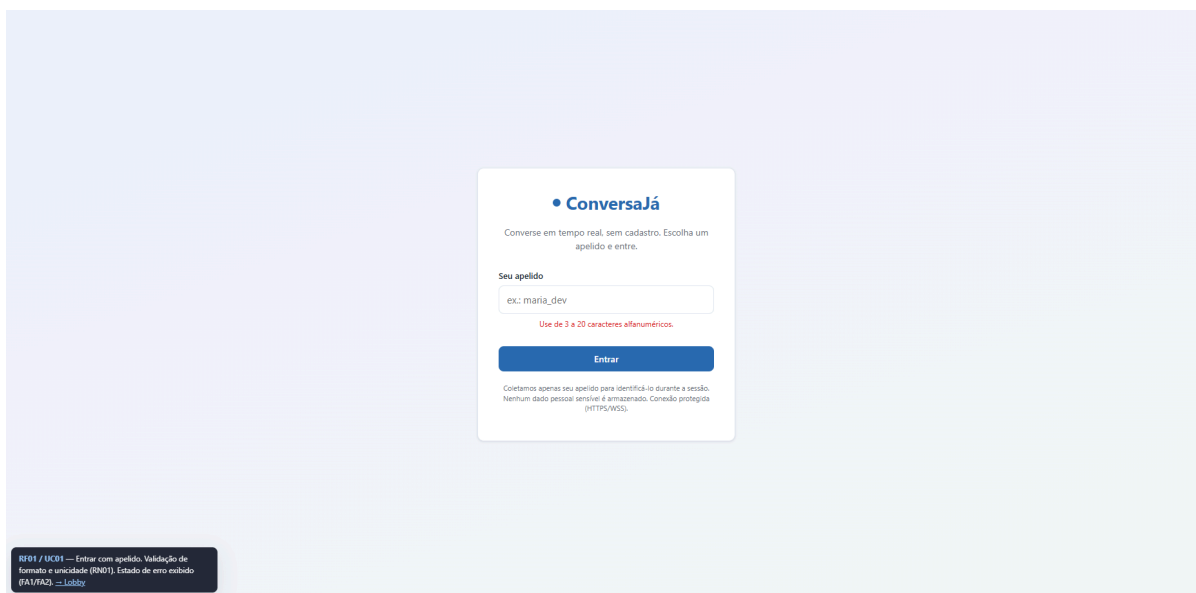


Figura 5 — Protótipo da tela de entrada (RF01).

7.2. Lobby de salas

Após entrar, o usuário vê o lobby com as salas públicas (nome, tema e número de participantes online), um campo de busca e a ação de criar uma nova sala. Salas oficiais recebem um selo distintivo. Ao criar uma sala, o usuário torna-se automaticamente seu moderador (RN03). Corresponde aos RF02 e RF04 / UC03.

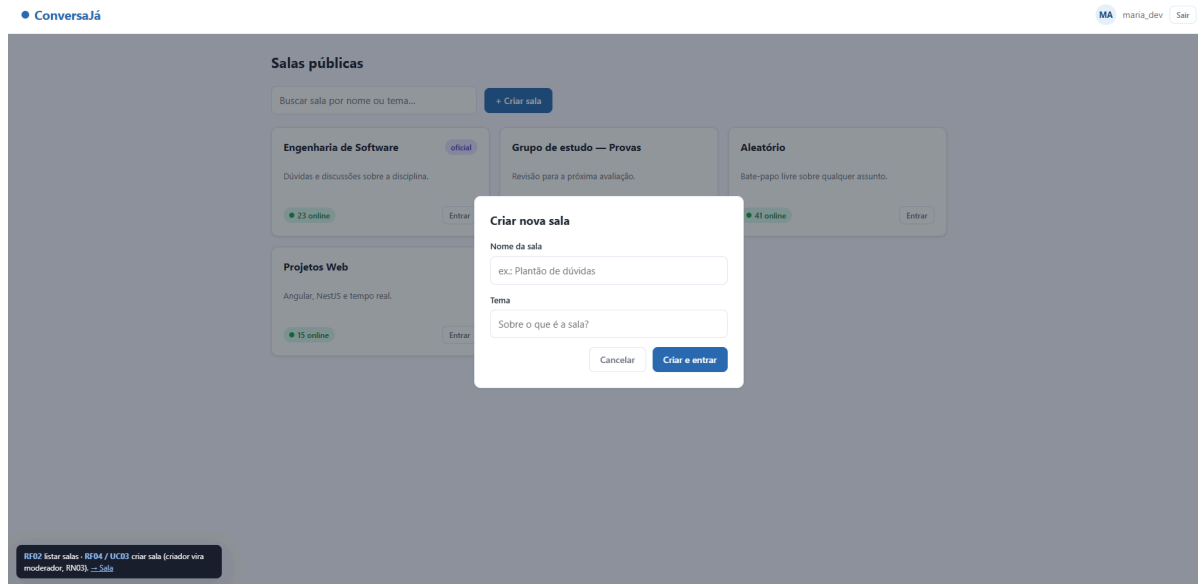


Figura 6 — Protótipo do lobby e da criação de sala (RF02, RF04).

7.3. Sala de conversa

A tela da sala concentra o núcleo do sistema: histórico recente de mensagens, avisos de entrada/saída, lista de participantes online, indicador de “digitando...”, contador de caracteres (limite de 500 — RN04) e campo de envio. Para o moderador, ficam disponíveis as ações de remover mensagem e expulsar participante (RF12/RF13, visíveis apenas a quem tem o papel — RN02). Cobre os RF03 e RF05–RF13.

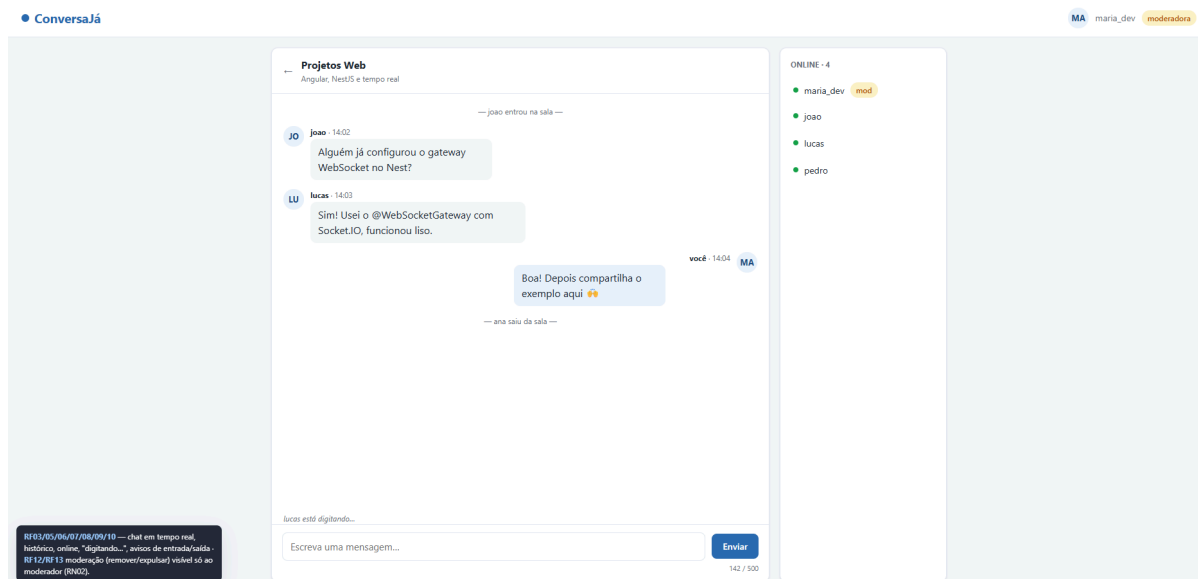


Figura 7 — Protótipo da sala de conversa em tempo real (RF03, RF05–RF13).

7.4. Painel administrativo

Acessível em /admin mediante token, o painel apresenta indicadores de uso (salas ativas, usuários online e mensagens na última hora) e o CRUD de salas oficiais; as alterações refletem-se no lobby de todos em tempo real. Cobre os RF14 e RF15.

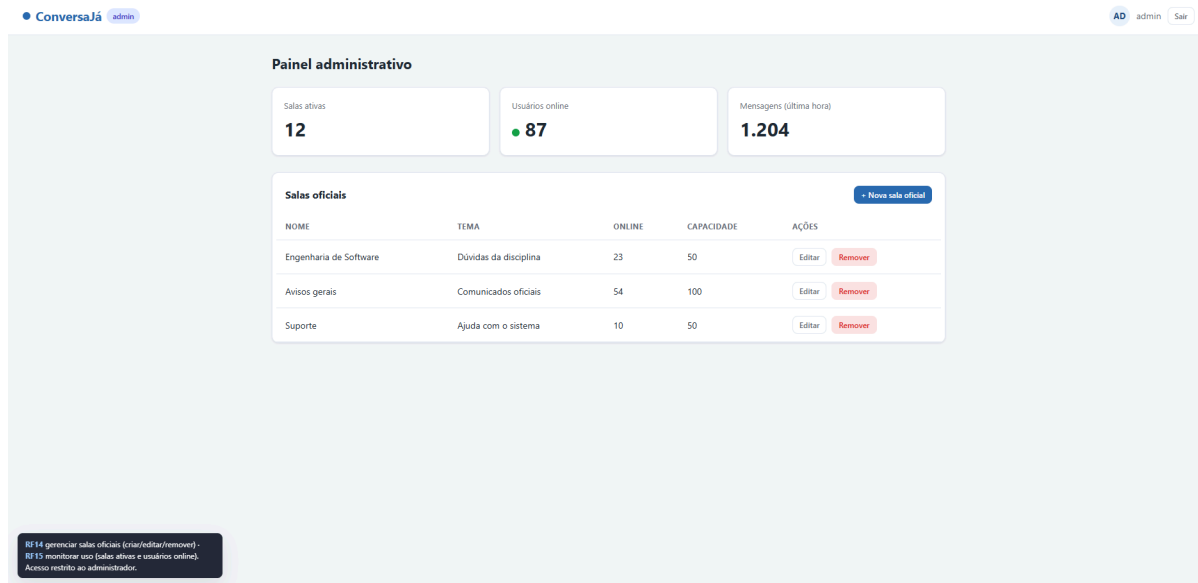


Figura 8 — Protótipo do painel administrativo (RF14, RF15).

8. Arquitetura do Sistema

8.1. Estilo arquitetural e justificativa

Adotou-se uma arquitetura monolítica modular: um único back-end, organizado internamente em módulos de domínio coesos e fracamente acoplados, em vez de microserviços. A decisão considera o escopo, o tamanho da equipe e a complexidade do sistema:

- O escopo gira em torno de uma funcionalidade central (chat em tempo real), e não de domínios independentes que justifiquem deploys separados.
- A equipe é pequena e o prazo é o da disciplina: um único artefato é mais simples de desenvolver, testar e implantar.
- Microserviços trariam custo de orquestração e de comunicação entre serviços, sem benefício real neste porte.

A modularidade interna preserva a manutenibilidade (RNF09): cada domínio evolui isolado, e o estado de tempo real fica concentrado no servidor, facilitando o broadcast. O código vive em um monorepo, o que permite compartilhar contratos entre cliente e servidor em um único lugar, evitando divergência.

8.2. Visão geral de componentes

O sistema é composto por um frontend Angular (SPA) que roda no navegador, um backend NestJS que expõe um gateway WebSocket (tempo real) e uma API REST (status e administração), um banco PostgreSQL para os dados duráveis e um pacote de contratos compartilhados usado pelos dois lados.

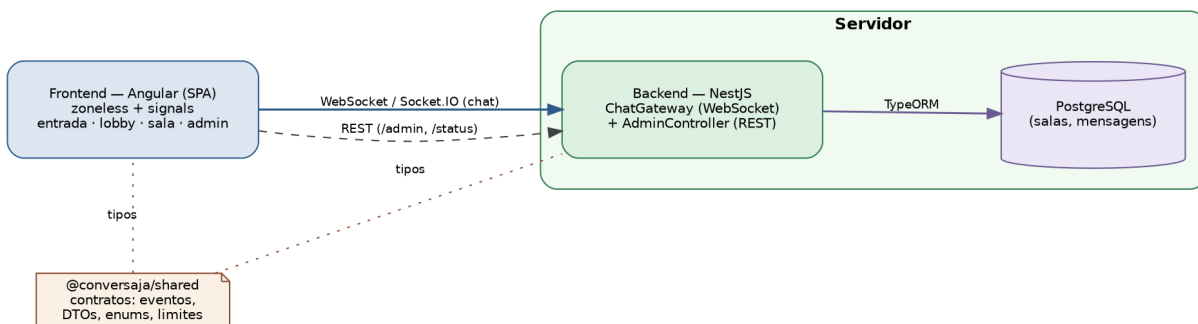


Figura 9 — Diagrama de componentes do ConversaJá.

8.3. Componentes principais

Frontend (Angular)

SPA com componentes standalone e reatividade baseada em signals (zoneless). Os componentes não falam diretamente com o socket: usam um SocketService tipado pelos contratos compartilhados.

Componente / serviço	Responsabilidade	Requisitos
features/entrada	Identificação por apelido.	RF01

Componente / serviço	Responsabilidade	Requisitos
features/lobby	Listar e criar salas.	RF02, RF04
features/sala	Chat em tempo real, online, “digitando...”, moderação.	RF03, RF05–RF13
features/admin	Painel administrativo (token, métricas, CRUD oficiais).	RF14, RF15
core/SocketService	Encapsula a conexão Socket.IO; expõe emit (com ack) e on (streams).	—
core/SessionService	Guarda o apelido da sessão.	RF01
core/authGuard	Impede acesso ao lobby/sala sem apelido.	RF01

Backend (NestJS)

Organizado em três módulos. O ChatModule é o núcleo de tempo real; o AdminModule trata da administração via REST; e o DatabaseModule cuida da conexão TypeORM/PostgreSQL.

Provider (ChatModule / AdminModule)	Responsabilidade
ChatGateway	Traduz eventos WebSocket em operações de domínio e faz o broadcast.
SessionService	Apelidos conectados e unicidade (RN01); total de usuários online (RF15).
RoomsService	Salas e presença online; RN03, RN06, RN07, RN08.
MessagesService	Conteúdo das mensagens; RN04, RN05 e sanitização contra XSS (RNF06).
MaintenanceService	Varredura periódica de salas ociosas (RN07).
AdminController / AdminService	Rotas /admin/* — métricas e CRUD de salas oficiais (RF14/RF15); dispara broadcast ao lobby após mudanças.
AdminGuard	Autenticação por token (x-admin-token / ADMIN_TOKEN).

Pacote compartilhado (@conversaja/shared)

Fonte única de verdade dos contratos: nomes de eventos WebSocket, formatos de payload, enums de domínio (Papel, Visibilidade, TipoMensagem) e os limites das regras de negócio (LIMITES, APELIDO_REGEX). É importado tanto pelo frontend quanto pelo backend, eliminando divergências de integração.

8.4. Formas de comunicação

A comunicação em tempo real é o coração do sistema: o cliente emite eventos e o servidor faz broadcast para os participantes da sala. As operações que precisam de confirmação usam o ack do Socket.IO, em que o handler devolve `{ sucesso: true, dados }` ou `{ sucesso: false, erro }`.

Direção	Evento	Finalidade
Cliente → Servidor	auth:entrar, sala:listar/criar/entrar/sair	Sessão e salas.
Cliente → Servidor	mensagem:enviar, mensagem:digitando	Envio e indicador de digitação.
Cliente → Servidor	moderacao:remover, moderacao:expulsar	Moderação.
Servidor → Cliente	mensagem:nova, sala:historico, sala:participantes	Conteúdo da sala.
Servidor → Cliente	sala:lista, sala:aviso, mensagem:digitando:status	Lobby e presença.
Servidor → Cliente	mensagem:removida, moderacao:expulso, erro	Moderação e erros.

Além do tempo real, há uma API REST: GET `/status` (saúde do serviço e limites de negócio) e `/admin/*` (métricas e CRUD de salas oficiais, protegido pelo AdminGuard). Em produção, o nginx serve a SPA e encaminha `/socket.io` para o backend, de modo que frontend e backend ficam na mesma origem — sem CORS e com tráfego sobre HTTPS/WSS (RNF06).

8.5. Fluxo crítico — envio de mensagem

O caminho crítico do sistema (RF05/RF06) é o envio de mensagem em tempo real. O cliente emite `mensagem:enviar`; o ChatGateway exige sessão e participação na sala; o MessagesService valida (RN04/RN05) e sanitiza (RNF06) o conteúdo, que é persistido no PostgreSQL; o remetente recebe o ack de sucesso e os demais participantes recebem a mensagem por broadcast (`mensagem:nova`). Em caso de conteúdo inválido, o servidor responde com ack de erro apenas ao remetente, sem broadcast.

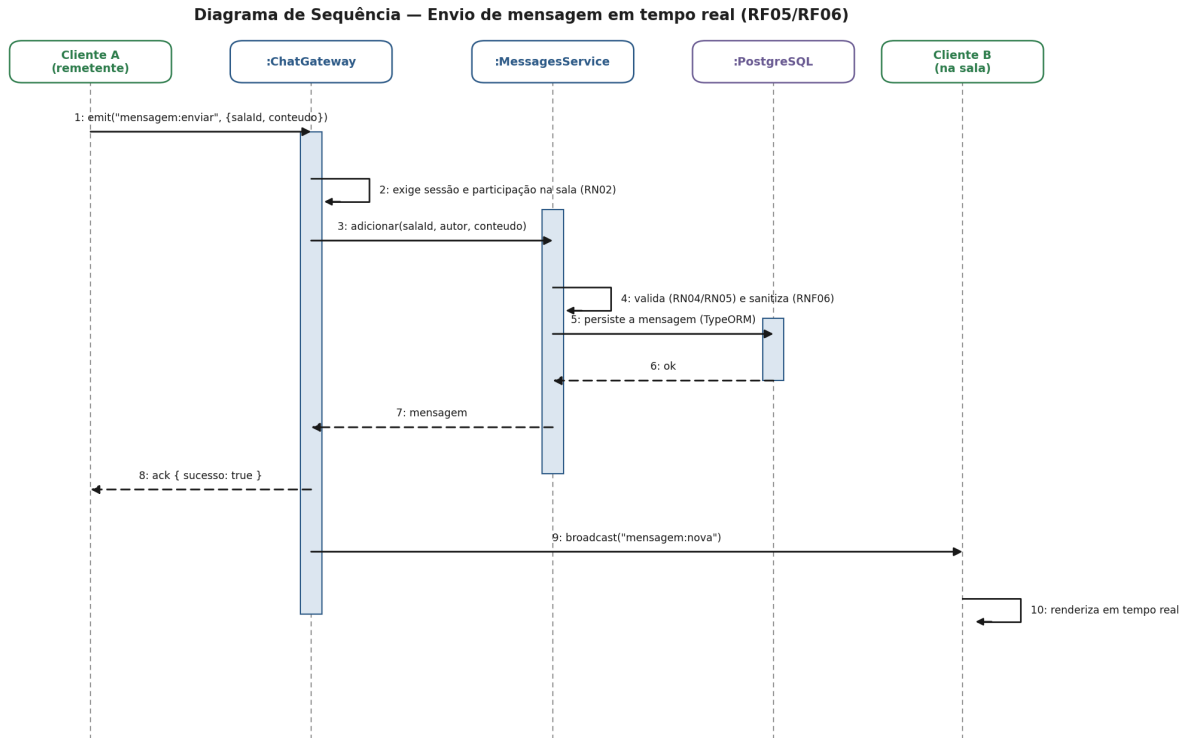


Figura 10 — Diagrama de sequência do envio de mensagem (RF05/RF06).

8.6. Persistência — durável vs. efêmero

Uma decisão central de modelagem foi separar o que é durável do que é efêmero. São duráveis (gravados em PostgreSQL via TypeORM) as salas e as mensagens. São efêmeros (mantidos em memória, atrelados às conexões) as sessões de apelido e a presença online por sala, além dos bloqueios temporários de reingresso (RN06) e do cronômetro de ociosidade (RN07). Isso é coerente com a natureza de tempo real: “quem está online” só faz sentido enquanto há conexão. O papel de moderador é derivado do criador da sala (RN03), persistido junto da sala.

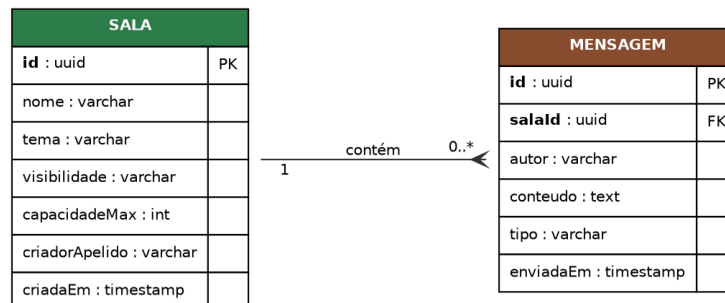


Figura 11 — Modelo de dados duráveis (entidades Sala e Mensagem).

O acesso a dados é abstraído pelo padrão de repositório (SalaStore e MensagemStore, classes abstratas usadas como tokens de injeção), com duas implementações: TypeORM em

produção e em memória nos testes. Assim, os testes unitários e o e2e de fluxo rodam sem banco, e trocar a tecnologia de persistência não afeta a lógica de domínio (RNF09).

8.7. Decisões arquiteturais relevantes

#	Decisão	Motivo
1	Monolito modular (não microsserviços)	Escopo de uma feature central; equipe pequena; menor custo operacional.
2	Monorepo (npm workspaces)	Compartilhar contratos cliente↔servidor numa fonte única.
3	WebSocket via Socket.IO	Tempo real bidirecional com rooms, acks e reconexão automática (RNF08).
4	TypeORM + PostgreSQL	Persistência madura em TypeScript, sem binários externos de engine.
5	Padrão de repositório (stores)	Testes sem banco e baixo acoplamento à persistência.
6	Presença/sessão em memória	Estado de tempo real é efêmero por natureza; simplicidade e desempenho.
7	Admin por token (não contas completas)	“Painel administrativo simples” do escopo, sem inflar complexidade.
8	Angular zoneless + signals	Modelo de reatividade moderno, simples para streams WebSocket.
9	nginx servindo a SPA + proxy WS	Mesma origem em produção; HTTPS/WSS; sem CORS.

9. Qualidade e Testes

A estratégia de testes cobre diferentes níveis, do unitário ao fluxo completo, priorizando as regras de negócio e o caminho crítico de tempo real. Ao todo, o projeto conta com cerca de 28 casos de teste automatizados, executados a cada alteração pelo CI.

9.1. Níveis de teste

Nível	Ferramenta	O que cobre
Unitário	Jest	Serviços e regras de negócio isoladas (RN01–RN08).
Integração	Jest + Nest TestingModule	Service ↔ repositório; validação de DTOs.
End-to-end	Jest + Socket.IO client	Fluxo real de dois clientes: entrar → criar/entrar → enviar → receber (RF01–RF07), sem banco.
Componente UI	Vitest + jsdom	Componentes Angular (lobby, sala, entrada), sem navegador.
Fluxo manual	Roteiro documentado	Validação ponta a ponta de um cenário real (ver 4.4).

9.2. Atributos de qualidade priorizados

A estratégia prioriza, de forma explícita, os atributos mais críticos ao produto e os relaciona aos requisitos do Trabalho 1:

- Corretude das regras de negócio — limites e permissões (RN01–RN08) têm cobertura unitária dedicada.
- Tempo real / desempenho — o fluxo envio → broadcast → recebimento (RF05/RF06, RNF01) é validado em teste e2e com múltiplos clientes simulados.
- Segurança — a sanitização de conteúdo contra XSS (RNF06) é testada com payloads maliciosos.
- Confiabilidade — a reconexão automática e a restauração de sessão (RNF08) são apoiadas pelo Socket.IO.

9.3. Casos de teste prioritários

- **auth**: apelido válido entra; fora do formato é rejeitado; em uso é rejeitado (RN01, UC01 FA1/FA2).
- **rooms**: criar sala adiciona à lista e torna o criador moderador (RN03); nome duplicado é rejeitado (UC03 FA2); sala cheia recusa entrada (RN08).
- **messages**: mensagem vazia ou acima de 500 caracteres é rejeitada sem broadcast (RN04, RN05, UC04 FA1); mensagem válida é persistida e retransmitida.
- **moderation**: não-moderador é negado ao remover/expulsar (RN02); expulso fica bloqueado por 10 minutos (RN06).

9.4. Teste de fluxo completo (end-to-end)

Há um teste de fluxo completo que valida o cenário central ponta a ponta: dois clientes WebSocket reais entram com apelidos distintos, um cria a sala e o outro entra, uma mensagem é enviada pelo primeiro e recebida pelo segundo em tempo real, e a lista de participantes reflete os dois. Por usar as stores em memória, o teste roda sem necessidade de banco de dados, cobrindo os RF01–RF07 em um único cenário automatizado.

9.5. Execução dos testes

```
npm run test      # unitários e integração (todos os workspaces)
npm run test:e2e  # fluxo WebSocket completo (backend)
npm run test:cov  # relatório de cobertura
```

10. Implantação e CI/CD

10.1. Implantação por terceiros (Docker Compose)

O sistema é implantável por qualquer pessoa com um único comando, sem configuração manual de ambiente. O `docker-compose.yml` define três serviços: db (PostgreSQL 16), backend (NestJS) e frontend (nginx servindo a SPA e encaminhando o WebSocket). O nginx coloca frontend e backend na mesma origem.

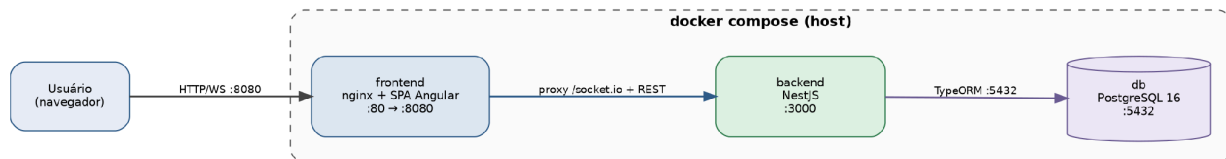


Figura 12 — Diagrama de implantação com Docker Compose.

Instruções de execução:

```

cp .env.example .env      # ajustar variáveis se necessário
npm run docker:up         # build + sobe db, backend e frontend
# acesse http://localhost:8080

npm run docker:logs       # acompanha os logs
npm run docker:down       # derruba os containers
  
```

O painel administrativo fica em `/admin` (ex.: `http://localhost:8080/admin`), autenticado pelo token definido em `ADMIN_TOKEN`. Todo o tráfego em produção deve ocorrer sobre HTTPS/WSS (RNF06).

10.2. Integração contínua (CI)

O repositório contém um pipeline de integração contínua no GitHub Actions (`.github/workflows/ci.yml`), disparado a cada push e pull request, que executa automaticamente as seguintes etapas — exigindo pipeline verde para o merge na branch principal:

#	Etapas	O que faz
1	<code>install</code>	Instala as dependências do monorepo (<code>npm ci</code>); compila o pacote <code>shared</code> .
2	<code>lint</code>	Análise estática (ESLint) e verificação de formatação (Prettier).
3	<code>test</code>	Testes unitários e de integração (backend e frontend).
4	<code>test:e2e</code>	Teste de fluxo completo via WebSocket.
5	<code>build</code>	Compila <code>shared</code> + backend + frontend, garantindo que o projeto é construível.

11. Rastreabilidade da Implementação

Cada funcionalidade implementada rastreia até um requisito definido no Trabalho 1, mantendo a continuidade entre especificação e código. Todas as funcionalidades estão concluídas.

RF	Funcionalidade	Onde (módulo / feature)
RF01	Entrar com apelido	backend chat/session · front entrada
RF02	Listar salas públicas	backend chat/rooms · front lobby
RF03	Entrar em uma sala	backend chat/rooms · front sala
RF04	Criar sala pública	backend chat/rooms · front lobby
RF05	Enviar mensagem em tempo real	backend chat/messages (gateway)
RF06	Receber mensagens em tempo real	front SocketService · sala
RF07	Visualizar usuários online	backend chat/rooms · front sala
RF08	Histórico recente	backend chat/messages
RF09	Indicador “digitando...”	backend gateway · front sala
RF10	Notificar entrada/saída	backend gateway
RF11	Sair da sala / sistema	backend gateway · front sala
RF12	Remover mensagem (moderação)	backend chat/messages (gateway)
RF13	Expulsar usuário (moderação)	backend gateway
RF14	Gerenciar salas oficiais (admin)	backend admin (REST) · front admin
RF15	Monitorar o sistema (admin)	backend admin · front admin

Regras de negócio implementadas: RN01 (apelido único, 3–20 alfanuméricos), RN02 (somente moderador modera), RN03 (criador vira moderador), RN04 (mensagens de até 500 caracteres), RN05 (sem mensagem vazia), RN06 (bloqueio de reingresso por 10 min após expulsão), RN07 (remoção automática de salas públicas ociosas) e RN08 (capacidade máxima da sala). Todas as RN01–RN08 estão cobertas.

12. Considerações Finais

O Trabalho 2 transformou o projeto conceitual do Trabalho 1 em um sistema Web funcional, testado e implantável, preservando a coerência com os requisitos, os protótipos e a stack definidos anteriormente. As 15 funcionalidades e as 8 regras de negócio do escopo foram implementadas; a arquitetura monolítica modular mostrou-se adequada ao porte do projeto; a estratégia de testes em múltiplos níveis, somada ao pipeline de CI, sustenta a qualidade a cada alteração; e a implantação por containers torna o sistema reproduzível por terceiros com um único comando. Entre os próximos passos, destacam-se a adoção de migrações versionadas no lugar do sincronismo automático de esquema, a ampliação dos testes de componente no frontend e a medição de desempenho sob carga (RNF01/RNF02).